

Generative AI & Large Language Models (LLMs)

AI Engineer Placement Preparation Series

Generated: February 2026 | Production-Ready Edition

Table of Contents

1. Why Generative AI & LLMs Matter
2. Pretraining Fundamentals
3. Instruction Tuning Deep Dive
4. Prompt Engineering Mastery
5. In-Context Learning & Few-Shot
6. Retrieval-Augmented Generation (RAG)
7. Fine-Tuning Large Models
8. Parameter-Efficient Fine-Tuning
9. Evaluation Metrics for LLMs
10. Agents & Tool Use
11. Safety & Alignment
12. Common Mistakes
13. Quick-Fire Q&A;
14. One-Line Cheat Sheet

1. Why Generative AI & LLMs Matter

If 2022 was "the year of BERT" and 2023 was "the year of ChatGPT," then 2024-2026 is the era of production-scale LLMs. Every major tech company (OpenAI, Google, Meta, Anthropic, Microsoft) is racing to deploy large language models. The bar for AI Engineer placements has never been higher.

What you MUST know: How LLMs are trained (pretraining + instruction tuning). How prompt engineering works and why it matters. How to fine-tune or deploy an LLM (LoRA, QLoRA, quantization). How to build systems on top of LLMs (RAG, agents, APIs). How to evaluate them and what their limitations are.

2025-2026 placement reality: Interviewers will ask you to explain scaling laws, describe instruction tuning, build a simple RAG system, or discuss the trade-offs of different fine-tuning approaches. Hands-on experience is mandatory.

2. Pretraining Fundamentals

2.1 The Pretraining Objective

LLMs are trained on massive unlabeled text corpora (Common Crawl, books, websites). The objective is simple: predict the next token given previous tokens. This self-supervised learning emerges into remarkable abilities.

2.2 Scaling Laws

Empirical finding (OpenAI, DeepMind): Loss decreases predictably with model size, dataset size, and compute. $\text{Loss} \propto 1/N^\alpha$ where N is scale (parameters, tokens, or compute), $\alpha \approx 0.07$. Doubling model size or data reduces loss by ~5%. This allows precise prediction of model performance before training.

2.3 Chinchilla Scaling (Optimal Compute Allocation)

Classic wisdom: scale model size more than data. Chinchilla (DeepMind 2022) shows optimal is: parameters $\approx$ tokens (both ~20x compute). GPT-3 (175B params, 300B tokens) is undertrained. LLaMA (70B params, 1T tokens) is better optimized. Implication: bigger isn't always better if undertrained.

2.4 Emergence & In-Context Learning

Emergence: Abilities arise unpredictably at scale (GPT-3 at 175B could do math, Code, etc). Smaller models can't. No clean predictor of what emerges when.

In-Context Learning (ICL): Models learn from examples in the prompt without gradient updates. "Translate to French: dog → chien. cat →" model outputs "chat". This is memorization + pattern matching in forward pass.

3. Instruction Tuning Deep Dive

3.1 Why Instruction Tuning?

Pre-trained models are raw: given a prompt, they complete it. "The capital of France is" → "the largest city in the world..." (continues indefinitely). They follow instructions poorly. Instruction tuning (fine-tuning on curated instruction-response pairs) makes them helpful assistants.

3.2 Data Format

```
# Instruction tuning data format (simplified):
{
  "instruction": "Summarize this text in one sentence.",
  "input": "The quick brown fox jumps over the lazy dog.",
  "output": "A fast fox jumps over a dog."
}

# Another format (chat):
[
  {"role": "user", "content": "Summarize in one sentence"},
  {"role": "assistant", "content": "A fast fox jumps over a dog."}
]

# Typical: 10K-100K instruction examples.
# Quality > quantity. 1K high-quality >> 100K low-quality.
```

3.3 Training Details

Freeze some early layers, train later layers + embedding. Use low learning rate (1e-5 to 5e-5). Early stopping on validation. Typical: 2-5 epochs. Overfit risk is high (small dataset vs huge model). Use regularization (dropout, weight decay).

3.4 Examples of Instruction Tuning

Type	Example	Use Case
Classification	"Is the sentiment positive? Text: I love this. Answer: Yes"	Sentiment, spam, topic
Generation	"Write a poem about winter." → [poem]	Creative, summaries
Q&A;	"Question: What's the capital of France? Context: [passage]"	Reading comprehension
Code	"Write Python code to sort a list."	Code generation, debugging

4. Prompt Engineering Mastery

4.1 Zero-Shot vs Few-Shot Prompting

Type	Format	When to Use
Zero-Shot	"Translate to French: hello"	Model knows task already
Few-Shot	"Translate: hello→bonjour. goodbye→au revoir. cat→?"	Teach by example. Better performance.

4.2 Chain-of-Thought (CoT) Prompting

Ask model to show reasoning step-by-step. "Q: If there are 3 apples and you add 2 more, how many total? Let me think step by step: ..." Models perform much better with explicit reasoning.

4.3 System Prompts vs User Prompts

System Prompt: Set overall behavior. "You are a helpful coding assistant. Always provide working code." Consistent across conversation.

User Prompt: Specific request. "Write a function to reverse a list." Changes each turn. System + user together define behavior.

4.4 Prompt Engineering Best Practices

1. Be specific: "list top 5 countries" vs "list countries" (vague)
2. Provide context: Background helps. "Given this tech stack, recommend..."
3. Show examples: Few-shot beats zero-shot
4. Specify output format: JSON, list, paragraph, etc.
5. Use role-playing: "Pretend you're a Python expert..."
6. Temperature tuning: Low (0.1-0.3) for deterministic. High (0.8-1.0) for creative.

5. In-Context Learning & Few-Shot

5.1 What Is In-Context Learning?

Model learns from examples in the prompt context window without updating weights. This is remarkable: the model "memorizes" patterns from examples and applies them. Larger models are better at ICL (scaling law).

5.2 Example Selection Matters

Random vs Semantic: Semantically similar examples >> random. If task is "translate English→French," examples should vary in difficulty/length, be high quality.

Order matters: Different orderings can change outputs by 5-10%. No universal best order. Mirror expected test distribution or use balanced ordering.

5.3 Context Window Limits

GPT-3.5: 4K tokens. GPT-4: 8K or 32K. Claude: 100K-200K (huge!). LLaMA: 4K. Larger windows allow more examples (better ICL) but increase inference cost ($O(n^2)$ attention). Practical: use as many examples as fit in budget.

6. Retrieval-Augmented Generation (RAG)

6.1 The Problem: Hallucinations & Knowledge Cutoff

LLMs hallucinate (confidently give wrong answers). Knowledge cutoff (trained on data up to date X, don't know recent events). Can't cite sources. Users can't verify claims.

6.2 How RAG Works

- 1. **Index:** Split documents into chunks. Embed chunks to vectors (OpenAI embeddings, sentence-transformers). Store in vector DB (Pinecone, Weaviate, FAISS).
- 2. **Retrieve:** User query → embed to same vector space → find top-k similar chunks from DB.
- 3. **Generate:** Use retrieved chunks as context. Feed to LLM: "Answer based on: [chunks]. Question: [query]"

Result: Answers grounded in documents. Can cite sources. Alleviates hallucinations.

6.3 Simple RAG Code

```
from sentence_transformers import SentenceTransformer
from pinecone import Pinecone
from openai import OpenAI

# 1. Index documents
embedder = SentenceTransformer('all-MiniLM-L6-v2')
chunks = ["Machine learning...", "Deep learning...", ...]
embeddings = embedder.encode(chunks)

pc = Pinecone(api_key="...")
index = pc.Index("my-index")
index.upsert(vectors=zip(range(len(chunks)), embeddings, chunks))

# 2. Retrieve
query = "What is machine learning?"
query_embedding = embedder.encode(query)
results = index.query(query_embedding, top_k=3)
context = "\n".join([r["metadata"]["text"] for r in results])

# 3. Generate
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "Answer using provided context."},
        {"role": "user", "content": f"Context: {context}\n\nQ: {query}"}
    ]
)
print(response.choices[0].message.content)
```

6.4 Advanced RAG Techniques

Technique	What It Does	When to Use
-----------	--------------	-------------

Reranking	Retrieve top-k, rerank with cross-encoder for quality	High recall needed
Hybrid Search	Combine semantic (vector) + keyword search	Mixed query types
Query Expansion	Generate multiple query variations, retrieve for all	Complex queries
Metadata Filtering	Filter chunks by date, source, category before retrieval	Specific domains

7. Fine-Tuning Large Models

7.1 When to Fine-Tune vs Prompt Engineering

Use Fine-Tuning When	Use Prompting When
Task-specific knowledge needed	General knowledge sufficient
Cost matters (cheaper per inference)	One-off queries
Latency critical	Standard latency ok
Privacy (keep data local)	Cloud API ok

7.2 Full Fine-Tuning (Rare for Large Models)

Update all weights. Requires huge GPU memory (GPT-3 is 175B params × 4 bytes = 700GB for weights alone, more for gradients). Impractical for models >13B. Use parameter-efficient methods instead.

7.3 Data Preparation

Quality >> quantity. Clean data, diverse examples, balanced labels. 100 high-quality examples >> 10K noisy examples. Test data should match distribution of deployment data.

8. Parameter-Efficient Fine-Tuning (PEFT)

8.1 Why PEFT?

Reduce trainable parameters 100-1000x while maintaining 97-99% of full fine-tuning performance. Enables fine-tuning on consumer GPUs. Standard practice now.

8.2 LoRA (Low-Rank Adaptation)

Instead of updating weight matrix W ($d_{in} \times d_{out}$), add trainable low-rank matrices A ($d_{in} \times r$) and B ($r \times d_{out}$). New output: $(W + AB) \times x$. Only A and B trained. r (rank) typically 8-16. Trainable params: $2 \times d_{in} \times r$ (vs $d_{in} \times d_{out}$).

```
# LoRA example: for every weight matrix in model:
# Before: h_new = W x h (no learnable params in this example)
# After: h_new = (W + ΔW) x h = W x h + A @ B @ h
# - W: pre-trained, frozen
# - A: [d_in, r], trainable
# - B: [r, d_out], trainable
# - r << min(d_in, d_out)

# GPT-3 (175B) with LoRA(r=8):
# - Full FT: 175B trainable params
# - LoRA: ~3.5M trainable params (0.002%)
# - Speed: 5x faster
# - Memory: 10x less
# - Quality: 97-99% of full FT
```

8.3 QLoRA (Quantized LoRA)

LoRA + 4-bit quantization. Model weights quantized to 4-bit (16x compression). Works on single GPU (RTX 4090). 70B model fits in 24GB. Slight quality loss (<1%) vs LoRA.

8.4 Comparison Table

Method	Trainable %	GPU Memory	Speed	Quality	When to Use
Full FT	100%	Huge	Slow	100%	Large GPU budget
LoRA	0.1-1%	Medium	Medium	98%	Default
QLoRA	0.1-1%	Low	Fast	97%	Limited GPU

9. Evaluation Metrics for LLMs

9.1 Automatic Metrics (Fast, Cheap)

Metric	Computes	Pros	Cons
BLEU	N-gram overlap	Fast	Doesn't capture semantics
ROUGE	Recall of n-grams	Summarization task	Variant-blind
METEOR	Precision+Recall+Synonyms	Better than BLEU	Still limited
BERTScore	Semantic similarity (embeddings)	Captures meaning	Computationally expensive

9.2 Human Evaluation (Gold Standard)

Experts rate outputs on quality, relevance, coherence (1-5 scale). Slow, expensive, but most reliable. Use for high-stakes decisions. Typically 100-500 samples, multiple annotators for consistency.

9.3 LLM-as-a-Judge

Use a strong LLM (GPT-4) to evaluate outputs. "Rate this summary 1-5: [summary]. [rubric]" Correlates well with human eval, much cheaper. Increasingly used in research.

9.4 Benchmarks

MMLU: Massive Multitask Language Understanding. 50+ domains, multiple choice. GPT-3: 65%, GPT-4: 88%, Claude: 87%.

HumanEval: Code generation. 164 problems, unit tests. Pass if code runs and passes tests. GPT-4: 92%, Claude: 88%, LLaMA: 25%.

HELM: Holistic Evaluation of Language Models. Comprehensive: accuracy, toxicity, bias, efficiency. Industry standard for responsible evaluation.

10. Agents & Tool Use

10.1 What Are LLM Agents?

LLMs can call tools (APIs, functions, calculators). Agent loop: think (reasoning) → act (call tool) → observe (result) → repeat. Enables solving complex tasks that require multiple steps.

10.2 Agent Loop

```
# Example: "What's the weather in Paris and NYC?"

# Agent loop:
query = "What's the weather in Paris and NYC?"

while not done:
    # 1. Think: LLM reasons about what to do
    thought = llm.reason(query)
    # "I need to check weather for two cities"

    # 2. Act: Decide which tool to call
    action = llm.decide_action(thought)
    # "Call get_weather('Paris') and get_weather('NYC')"

```
 # 3. Observe: Execute tool, get results
 observation = execute_action(action)
 # {"Paris": "15°C, sunny", "NYC": "20°C, rainy"}

 # 4. Update context with observation
 context.append((thought, action, observation))

 # 5. Check if done
 if llm.check_if_done(context):
 done = True

Final: LLM generates response using all context
response = llm.generate(context)
"Weather in Paris: 15°C sunny. Weather in NYC: 20°C rainy."
```


```

10.3 Tool Use Approaches

Approach	Format	Model Support
Function Calling	API returns structured tool calls	GPT, Claude, Gemini
ReAct	Text output "[TOOL_NAME] args\n[OBSERVATION]"	Any LLM via prompting
JSON Mode	Model outputs valid JSON with tool specs	GPT-4, some open models

10.4 Frameworks

LangChain: Orchestrate agents, chains, memory. Large ecosystem. Popular but complex.

LlamaIndex: Data indexing, RAG pipelines. Great for retrieval-based agents.

Anthropic's MCP: Model Context Protocol. Emerging standard for tool use. Standardized tool definitions.

11. Safety & Alignment

11.1 Common Alignment Issues

Hallucinations: Confident false statements. Mitigation: retrieval (RAG), uncertainty quantification.

Bias: Reflects training data biases (gender, racial stereotypes). Mitigation: diverse training data, red-teaming.

Toxicity: Generates harmful content. Mitigation: instruction tuning on safe data, classifiers to filter.

Misuse: Used for abuse, deception, etc. Mitigation: access controls, usage monitoring, usage policies.

11.2 RLHF (Reinforcement Learning from Human Feedback)

1. Collect human preferences: annotators rank outputs (A vs B). 2. Train reward model: predict human preference. 3. Use RL (PPO) to optimize model outputs for high reward. Makes models safer, more helpful. Used in ChatGPT, Claude.

11.3 DPO (Direct Preference Optimization)

Simpler alternative to RLHF. Don't train reward model explicitly. Directly optimize LLM to prefer good outputs over bad (preference pairs). Easier, more stable training. Increasingly used.

12. Common Mistakes

Ignoring context window limits: 4K-8K tokens fills up fast. Truncating important context hurts performance.

Using wrong temperature: 0.0 for deterministic, 1.0 standard, 1.5+ very random. High temp for creative, low for facts.

Not versioning prompts: Prompt changes affect output. Version control prompts like code.

Overfitting on small fine-tune datasets: Model memorizes. Use regularization, early stopping.

Ignoring hallucinations in production: Always fact-check. Use RAG. Monitor outputs.

Not using few-shot examples: Few-shot almost always beats zero-shot. Minimal cost.

Training on biased data: Model learns and amplifies bias. Clean and audit data.

Forgetting inference optimization: Quantization, distillation, batch inference reduce cost/latency 2-10x.

Not monitoring in production: Model degrades over time (data drift). Monitor performance, retrain periodically.

Treating LLMs as magic: They're powerful but fallible. Understand limitations. Test thoroughly before deployment.

13. Quick-Fire Q&A;

[Easy] What's the difference between pretraining and fine-tuning?

Pretraining: learn language on massive unlabeled data. Fine-tuning: specialize to task using labeled data.

[Easy] Why does few-shot learning work?

Model learns patterns from examples in context. In-context learning: weights frozen, patterns from prompt.

[Medium] Explain instruction tuning in one sentence.

Fine-tune on (instruction, response) pairs to teach model to follow instructions, not just predict next token.

[Medium] What's the computational cost difference between pretraining and fine-tuning?

Pretraining: weeks-months, millions of dollars. Fine-tuning: hours-days, thousands of dollars. 100-1000x difference.

[Hard] Why is LoRA so effective?

Weights change slightly during fine-tuning. LoRA captures this via low-rank perturbation. Full rank unnecessary.

[Medium] When should you use RAG vs fine-tuning?

RAG: need external docs, knowledge cutoff issue, quick updates. Fine-tuning: task-specific patterns, cheaper inference.

[Medium] What's the difference between zero-shot and few-shot?

Zero-shot: no examples, just task description. Few-shot: examples in context. Few-shot usually 20-50% better.

[Hard] How does emergence work? Why do small models not show abilities of large models?

Unknown. Might be: task complexity threshold, memorization vs generalization transition. Active research area.

[Medium] What's the difference between temperature and top-k sampling?

Temperature: scales logits. Top-k: sample from top-k tokens only. Both add diversity vs greedy.

[Medium] How do you prevent hallucinations?

RAG grounds in docs. Use chain-of-thought. Uncertainty quantification. Verify with external sources.

14. One-Line Cheat Sheet

- Pretraining — Train on massive unlabeled text via next-token prediction. Emerges language understanding.
- Scaling laws — $\text{Loss} \propto 1/N$. Doubling data/model size reduces loss ~5%. Predictable improvement.
- Emergence — Abilities appear at scale (175B) that don't exist at 7B. Unpredictable which abilities emerge when.
- Instruction tuning — Fine-tune on curated (instruction, response) pairs. Teaches following instructions.
- In-context learning — Learn from examples in prompt context without weight updates. Larger models better.
- Few-shot prompting — Provide examples in prompt. Usually 20-50% better than zero-shot.
- Chain-of-thought — Ask model to show step-by-step reasoning. Improves complex reasoning.
- System prompt — Set overall behavior at conversation start. User prompt is specific request.
- RAG — Retrieve relevant documents, feed to LLM. Alleviates hallucinations, adds citations.
- LoRA — Add low-rank trainable matrices to frozen weights. 0.1-1% trainable params, 97-99% quality.
- QLoRA — LoRA + 4-bit quantization. Fit 70B model on single GPU. Practical for consumer hardware.
- BLEU/ROUGE/METEOR — Automatic metrics based on n-gram overlap. Fast but limited (no semantics).
- BERTScore — Semantic similarity via embeddings. Better than BLEU, more expensive.
- Human evaluation — Annotators rate 1-5. Slow expensive but gold standard. Use for high-stakes.
- LLM-as-a-judge — Use GPT-4 to evaluate. Correlates with human eval, much cheaper.
- Agent loop — Think (reasoning) → Act (call tool) → Observe (result) → repeat. Solves multi-step tasks.
- Function calling — Model returns structured tool calls. Cleaner than text-based ReAct.
- RLHF — Train on human preferences. Use reward model + RL (PPO). Makes model helpful and safe.
- DPO — Direct preference optimization. Simpler than RLHF. No explicit reward model.
- Temperature — 0.0 deterministic, 1.0 standard, >1.0 random. Control output diversity.

Final Note

LLMs are the most impactful AI advancement in years. Understanding how they work—pretraining, instruction tuning, fine-tuning, prompting, RAG—is essential for 2025-2026 placements. The field moves fast: new models, techniques, benchmarks emerge constantly. Build projects: fine-tune a model, create an agent, build a RAG chatbot. Read papers on arXiv (transformers, LLM, RLHF). Follow leaders: OpenAI, Anthropic, Meta, Google, Hugging Face. Understand limitations: hallucinations, bias, cost. You're living in the LLM era. Master it.